



Puzzle editor and solver

A*

BFS

DFS

rushhour write-up

February 13th, 2025

© 2025 Paul Mancion. Please do not redistribute without permission.

<https://github.com/MindMidas/rushhour>

How does the Brute Force, Breadth-First-Search Algorithm Operate

The BFS algorithm explores all possible paths level by level. This ensures it finds the shortest path to the solution in the Rush Hour Problem. It uses a queue (FIFO struct) to expand the shallowest unvisited nodes before moving deeper in the graph. Each node has a path which contains visited states to prevent looping. The unique states are tracked, and a check was implemented to avoid exploring previously visited states. This optimized the search significantly, reducing memory usage.

How does the Best-First-Search Algorithm Operate

The Best-FS algorithm selects the most promising node using one of the three heuristics implemented. Essentially, it prioritizes states that are likely to lead to the solution. A priority queue (heap) was used to sort nodes based on their heuristic values.

The evaluation function: $f(n) = h(n) + g(n)$, ensures that the algorithm will find the shortest path as it will explore a previously generated state if the cost gets too high.

How the Results were Generated

The results were generated by running BFS and Best-FS (heuristic-1, heuristic-2, and heuristic-3) algorithms on the Rush Hour problem and the results were logged in a results.json file which was used to store and extract the data for the graphs and charts. These algorithms traversed the problem state spaces, measured the number of states visited, depths to solutions, and non-looping paths. In addition, each algorithm was run on 40 puzzles and everything ran smoothly.

Heuristics Implemented

Heuristic 1:

Measures and returns the distance of the red car (X) to the exit. It calculates how many moves are needed for the red car's leftmost coordinate to reach the right edge of the grid (4,2).

Admissibility Proof for $h_1(n)$:

1. Let,

$x_{car} = [(x_1, y_1), (x_2, y_2)]$, the red car inside the grid of a Rush Hour Problem.

2. Assuming there are no horizontal cars blocking x_{car} path to *goal*:

Let,

$goal = [(4, 2), (5, 2)]$, the solution coordinate for x_{car} inside the grid of a Rush Hour Problem.

3. We can now define $h_1(n)$:

$$h_1(n) = goal[0][0] - x_{car}[0][0]$$

4. True Cost $h^*(n)$:

The actual cost to reach the shortest solution state is at least the number of moves required to get:

$x_{car} = goal$, plus, any additional moves needed to move vehicles blocking the x_{car} path.

5. Since the following inequality holds in the Rush Hour Problem:

$$0 \leq (x_{car}[0][0]) \leq 4$$

The $h_1(n)$ represents the minimum number of moves needed for the x_{car} to reach the *goal*. The actual cost $h^*(n)$ is at least $h_1(n)$ plus any additional moves required to clear blocking vehicles.

6. Since $h_1(n)$ does not account for blocking vehicles, it is not necessarily an optimal cost, so it may underestimate the actual cost to reach the *goal* but it will never overestimate the true cost.

The following inequality is established:

$$h_1(n) \leq h^*(n)$$

7. Thus, $h_1(n)$ is a lower bound, ensuring admissibility.

Heuristic 2:

This heuristic builds on heuristic 1 by adding the total number of vehicles blocking the red car's (X) path. It considers both the red car's distance to the goal state and how many other vehicles must be moved to clear the path. This gives us a slightly more accurate cost to reaching the goal when compared to heuristic 1.

Admissibility Proof for $h_2(n)$:

Building on $h_1(n)$ for $h_2(n)$:

1. Let,

$vehicles = \{v_1, v_2, \dots, v_n\}$, where each v_i is a vehicle inside the grid of a Rush Hour Problem.
 $x_{car} = [(x_1, y_1), (x_2, y_2)]$, the red car inside the grid of a Rush Hour Problem.

2. Assuming there are no horizontal cars blocking x_{car} path to goal:

Let,

$$v \in vehicles$$

If v is a car, let,

$$v_{car} = [(x_1, y_1), (x_2, y_2)], \text{ where } v_{car}[0] \text{ is the top leftmost coordinate.}$$

If v is a truck, let,

$$v_{truck} = [(x_1, y_1), (x_2, y_2), (x_3, y_3)], \text{ where } v_{truck}[0] \text{ is the top leftmost coordinate.}$$

3. To determine whether a vehicle is blocking x_{car} path to goal, we define the blocking function:

if $v_{car} \neq x_{car}$:

$$blocking(v_{car}, x_{car}) = \begin{cases} 1 & | \ v_{car}[i][1] = x_{car}[1][1] \text{ for any } i \in \{0, 1\} \\ 0, & \text{Otherwise.} \end{cases}$$

$$blocking(v_{truck}, x_{car}) = \begin{cases} 1 & | \ v_{truck}[i][1] = x_{car}[1][1] \text{ for any } i \in \{0, 1, 2\} \\ 0, & \text{Otherwise.} \end{cases}$$

A vehicle v is blocking x_{car} if any part of v shares the same y – coordinate as x_{car} .

4. We can now define $h_2(n)$ function:

$$totalBlocking = \sum_{i=1}^{len(vehicles)} blocking(v_i, x_{car})$$

$$h_2(n) = h_1(n) + totalBlocking$$

5. True Cost $h^*(n)$:

The actual cost includes:

- The distance of the x_{car} to the exit which is at least $h_1(n)$.
- The $totalBlocking$ value is a lower bound on the minimum number of moves required to clear blocking vehicles, ensuring that $h^*(n)$ accounts for all necessary moves.

The following inequality is established:

$$h^*(n) \geq h_1(n) + totalBlocking$$

6. Since $h_2(n)$ never overestimates but may underestimate the true cost, the following inequality holds:

$$h_2(n) \leq h^*(n)$$

7. Thus, $h_2(n)$ is admissible.

Heuristic 3:

This heuristic builds on heuristic 1 and improves on heuristic 2. It considers both the red car's distance to the goal state and the actual cost to move the blocking vehicles. It is not recursively counting the cost to move vehicles blocking vehicles as this would end up consuming more time. Instead, it assigns a move cost that is based on the vehicle type (car or a truck) and the minimum amount of moves needed to free the red car's path by going up or down. This heuristic provides a better estimate of the true cost to solve the puzzle.

Admissibility Proof for $h_3(n)$:

Building on $h_1(n)$ while using $blocking()$ from $h_2(n)$ for $h_3(n)$:

1. Let,

$x_{car} = [(x_1, y_1), (x_2, y_2)]$, the red car inside the grid of a Rush Hour Problem

2. Assuming there are no horizontal cars blocking x_{car} path to *goal*:

We will be using the same definition of *vehicles*, v , v_{car} , and v_{truck} and the $blocking()$ function from $h_2(n)$ to define the following vehicles blocking function to find the vehicles in the path of the x_{car} :

$$blockingVehicles(x_{car}) = \{v_i \mid i = 1, 2, \dots, len(vehicles), blocking(v_i, x_{car}) = 1\}$$

3. Next, we define the cost to move up:

Let,

$$costUp(v_{car}, x_{car}) = \begin{cases} 1 & | v_{car}[1][1] = x_{car}[1][1] \\ 2 & | v_{car}[0][1] = x_{car}[1][1] \end{cases}$$
$$costUp(v_{truck}, x_{car}) = \infty, \text{ since there's insufficient space in the grid to move } v_{truck} \text{ up.}$$

4. Next, we define the cost to move down:

Let,

$$costDown(v_{car}, x_{car}) = \begin{cases} 1 & | v_{car}[0][1] = x_{car}[1][1] \\ 2 & | v_{car}[1][1] = x_{car}[1][1] \end{cases}$$
$$costDown(v_{truck}, x_{car}) = \begin{cases} 1 & | v_{truck}[0][1] = x_{car}[1][1] \\ 2 & | v_{truck}[1][1] = x_{car}[1][1] \\ 3 & | v_{truck}[2][1] = x_{car}[1][1] \end{cases}$$

5. Now we define the blocking cost:

Since each blocking vehicle must move to clear the path of x_{car} the blocking cost is now:

$$blockingCost = \sum_{v \in blockingVehicles(x_{car})} \min(costUp(v, x_{car}), costDown(v, x_{car}))$$

6. True Cost $h^*(n)$:

The actual cost includes:

- The distance of the red car to the exit which is at least $h_1(n)$.
- The $blockingCost$ value is a lower bound on the minimum number of moves required to clear blocking vehicles, ensuring that $h^*(n)$ accounts for all moves needed to clear the path each v in $blockingVehicles$.

The following inequality is established:

$$h^*(n) \geq h_1(n) + blockingCost$$

7. Since $h_3(n)$ never overestimates but may underestimate the true cost, the following inequality holds:

$$h_3(n) \leq h^*(n)$$

8. Thus, $h_2(n)$ is admissible.

Admissibility Analysis:

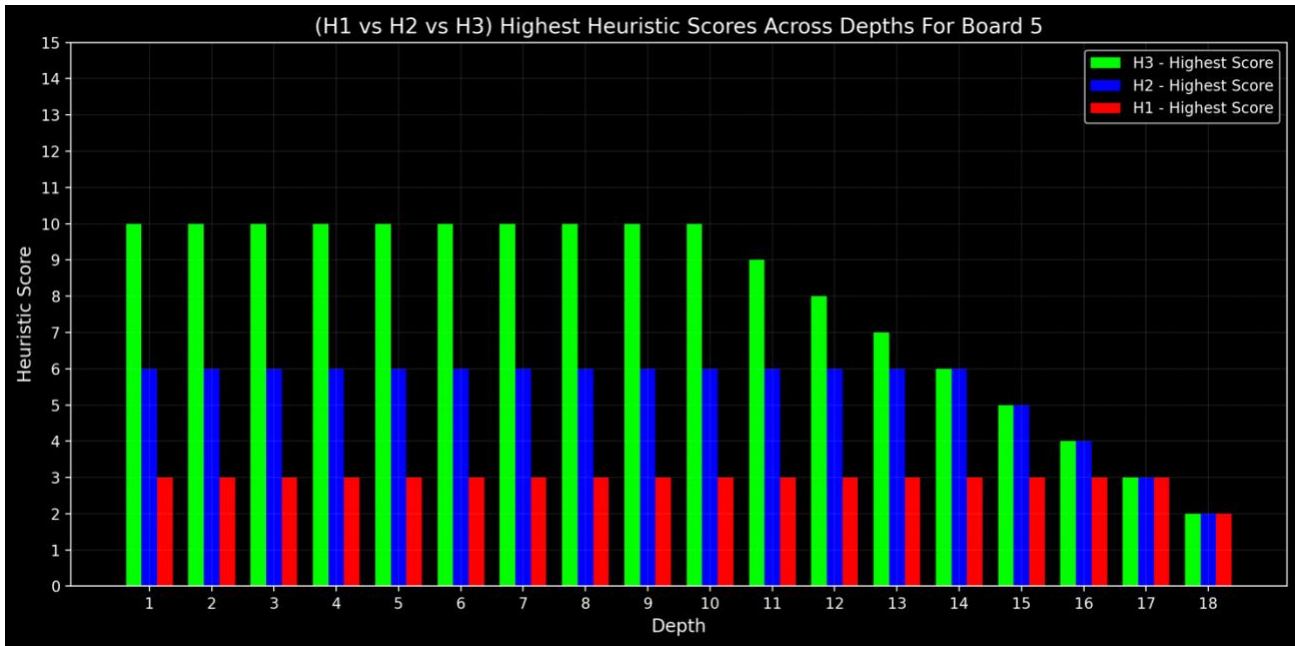


Figure 1: Highest Heuristic Scores Across Depths for Board 5

In Figure 1, the data used to create the graph represents the highest heuristic score across all states at each depth for the Best-FS heuristics (H1, H2, and H3) on Board 5. This graph supports the argument that H1, H2, and H3 are all admissible because none of the heuristic estimates exceed the actual solution depth. The shortest non-looping path to the solution occurs at depth 18. The highest recorded heuristic value for H1 is 3, for H2 is 6, and for H3 is 10—all of which remain below the solution depth. This confirms that each heuristic does not overestimate the true cost to the solution, maintaining admissibility.

In addition, an extra admissibility check was implemented to ensure that the heuristic values never exceed the optimal solution depth found using BFS. Since BFS guarantees the shortest path, the solution depth was used as a benchmark for admissibility. Using the 40 provided puzzles, BFS was run on each one to determine the solution depth. The highest heuristic scores from each heuristic (H1, H2, and H3) were then compared to these solution depths for the same 40 puzzles to verify admissibility.

Comparing Heuristic Estimates

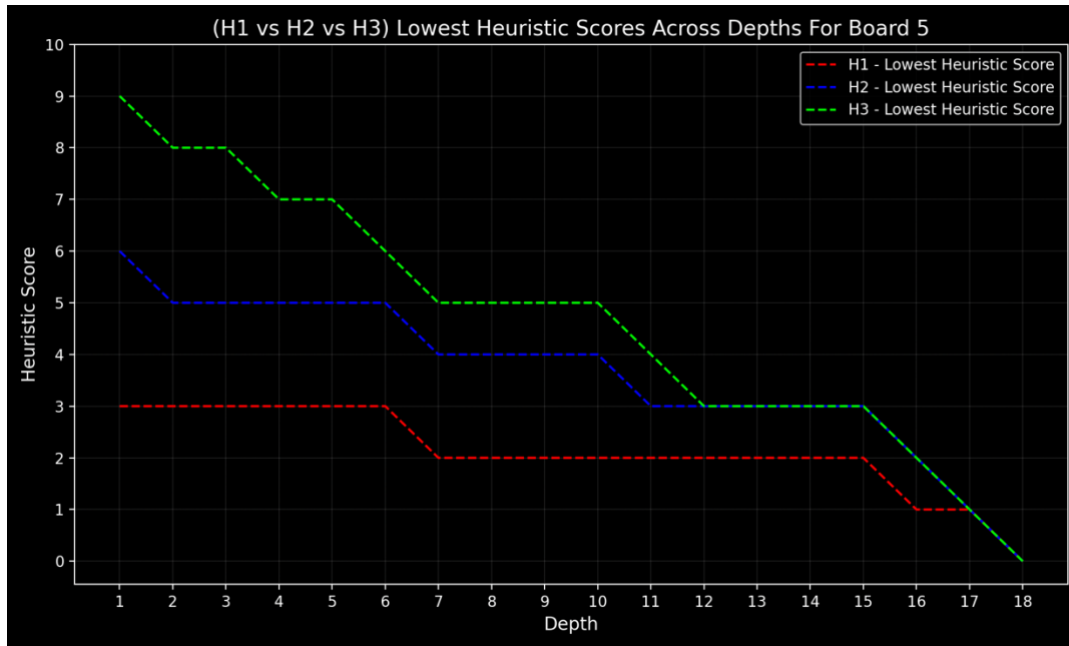


Figure 2: (H1 vs H2 vs H3) Lowest Heuristic Scores Across Depths for Board 5

In Figure 2, the data that was used to create the graph was the lowest heuristic score across all scores generated for each state at a given depth for each Best-FS heuristic (H1, H2, and H3) on Board 5. The graph shows that H3 consistently provides the highest, and occasionally equally high, heuristic estimates across different depths when compared to H2 and H1. It also shows that H2 provides higher, and occasionally equally high heuristic estimates when compared to H1.

The most basic heuristic is H1, it only considers red car's distance to the goal. This means it ignores blocking vehicles, making it an admissible but a weak estimate. H2 builds on H1 by counting the number of vehicles blocking the red car's path to the goal. This makes H2 a better heuristic than H1 as it accounts for obstacles in the path and the red car's distance to the exit. Finally, H3 build on H1 and improves on H2 by estimating the minimum number of moves required to reposition each of the vehicles blocking the red car's path to the goal. In addition, H3 also considers the red car's distance to the goal. This gives a more realistic estimate of the actual cost to reach the goal.

All heuristic estimates eventually converge (depth 17 in Figure 2) to the same final value near the solution depth, it supports the claim of admissibility, as they do not exceed the true cost to the goal. This is further reinforced with Figure 3 below.

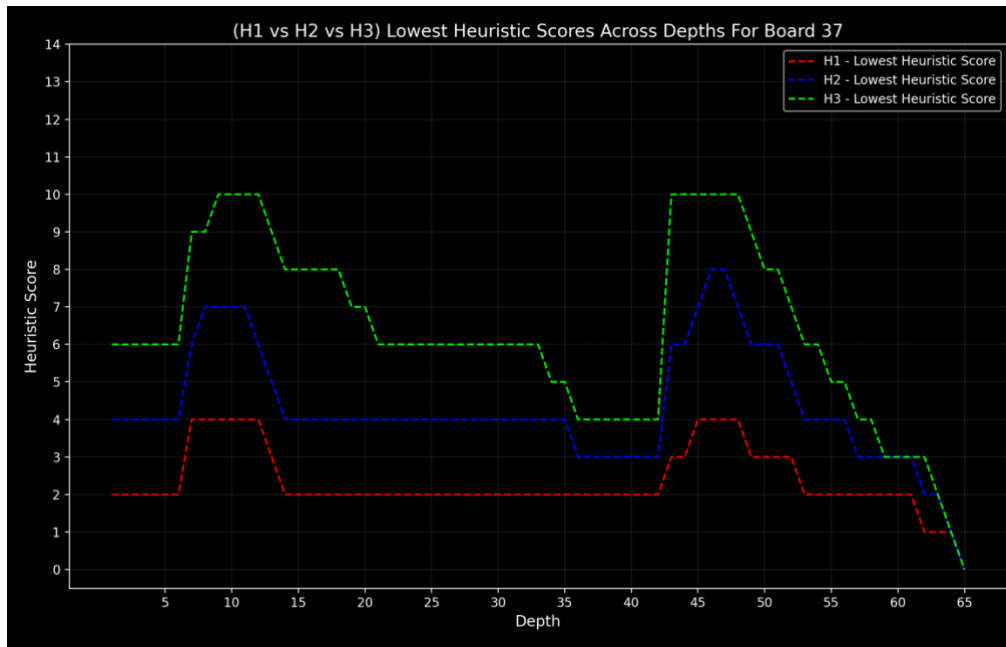


Figure 3: (H1 vs H2 vs H3) Lowest Heuristic Scores Across Depths for Board 37

The same method that was used in Figure 2 was used for Figure 3, but this time on Board 37. This graph is interesting because it shows that H1, H2, and H3 heuristic scores are not always perfectly decreasing. At depths 5 and 43, the Best-FS algorithm for all heuristics is forced to revisit previous states due to the $g(n)$ property. It then returns to the current depth (5 and 43) and realizes that the best path forward is a board which temporarily appear worse. This leads to a higher heuristic estimate (as seen in the sudden spikes in Figure 3), which then plateaus, and quickly drops as a better path emerges. This behavior occurs because my heuristics only estimate the cost of moving forward making the board look “better” instead of “worse”.

It's also worth noting that in both Figure 2 and Figure 3, as the search progresses and states get closer to the goal, the additional complexity of H3 and H2 does not always result in a higher heuristic estimate. This can be seen where the heuristic lines overlap. It indicates that the estimates are equal at certain depths.

Unique State Coverage by Depth

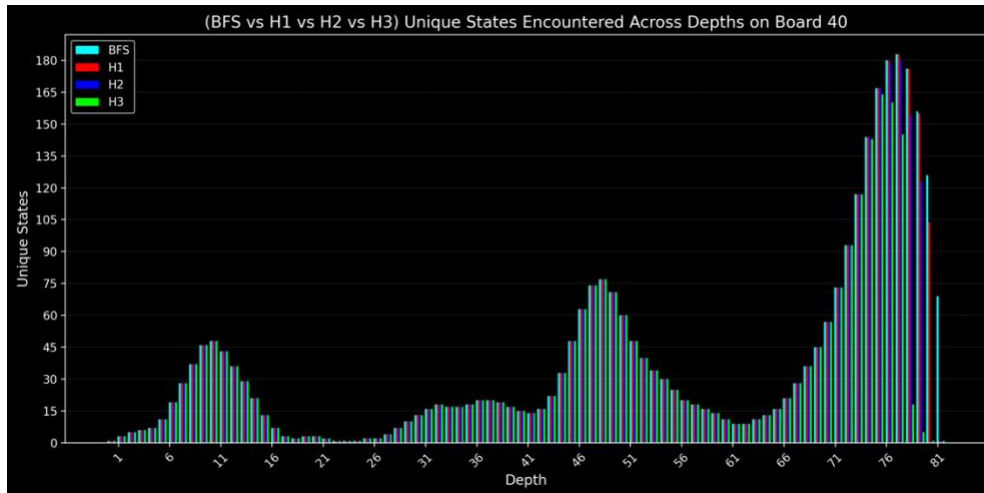


Figure 4: BFS vs H1 vs H2 vs H3) Unique States Encountered Across Depths on Board 40

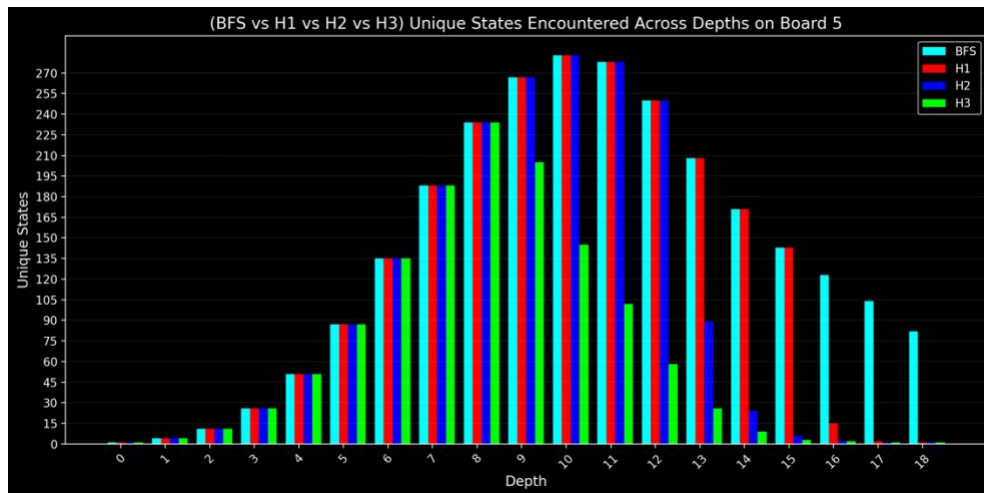


Figure 5: BFS vs H1 vs H2 vs H3) Unique States Encountered Across Depths on Board 5

Figures 4 and 5 display the number of unique states encountered across different depths for Board 40 and Board 5, respectively, comparing BFS, H1, H2, and H3. There's a consistent trend in both graphs, where the overall shape of the curves—peaks and dips—remains similar across all algorithms. In Figure 5, BFS visits the most unique states at all depths. This is expected from a brute-force algorithm. Up to depth 8, all the algorithms are closely tied, but then H3 begins to drop, followed by H2 at depth 13, and H1 at depth 16. This pattern reinforces the argument that H3 produces the highest heuristic estimates, followed by H2 and then H1, visiting fewer unique states as the solution approaches.

A similar trend is observed in Figure 4. In the end stages of the search, H3, H2, and H1 deviate from BFS in the same order. However, Figure 4's board is of significantly higher difficulty when compared to Figure 5's board. This suggests that as puzzle difficulty increases, the heuristics become less effective relative to BFS, until they start nearing the end of the search and cost estimations improve.

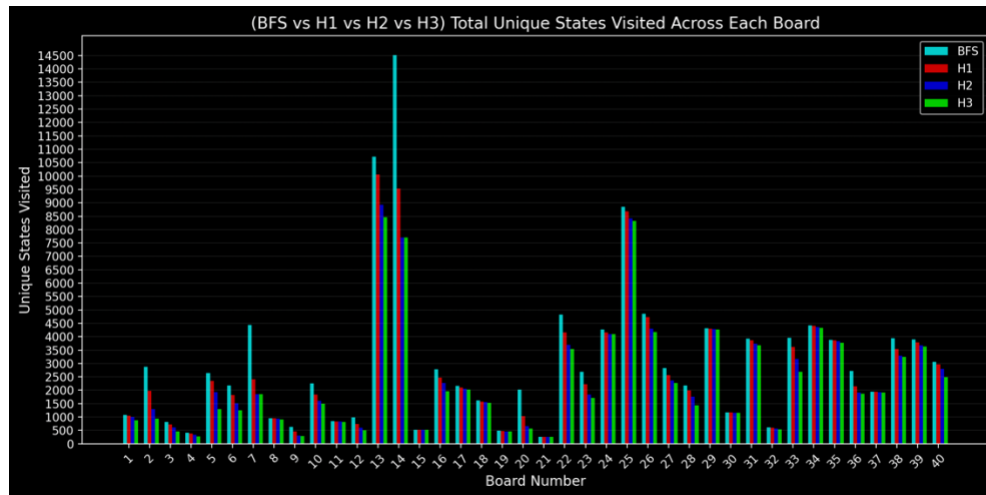


Figure 6: BFS vs H1 vs H2 vs H3) Total Unique States Visited for Each Board

Figure 6 compares the total unique states visited by BFS, H1, H2, and H3 across the 40 different boards. As expected, BFS explores the most unique states, and on some occasions by a significant margin (boards 2, 7, 14, 20). H3 visits the fewest number of unique states, demonstrating the estimation's ability to guide the search more efficiently. H1, being the simplest heuristic, does not significantly reduce the number of visited states compared to BFS. The variation in unique state coverage across all the different boards highlights how puzzle complexity influences search performance. Some puzzles require far more exploration compared to others even though the difficulty isn't as high. This graph shows that heuristics become less effective as the problem difficulty and depth increases. Essentially, as we approach the most difficult puzzles on the right end of the graph, the number of unique states visited by each algorithm in comparison to BFS is much closer. Furthermore, there are a few bar spikes which highlight how search complexity escalates as the problem space grows.

Solution Depth and Time Taken Analysis

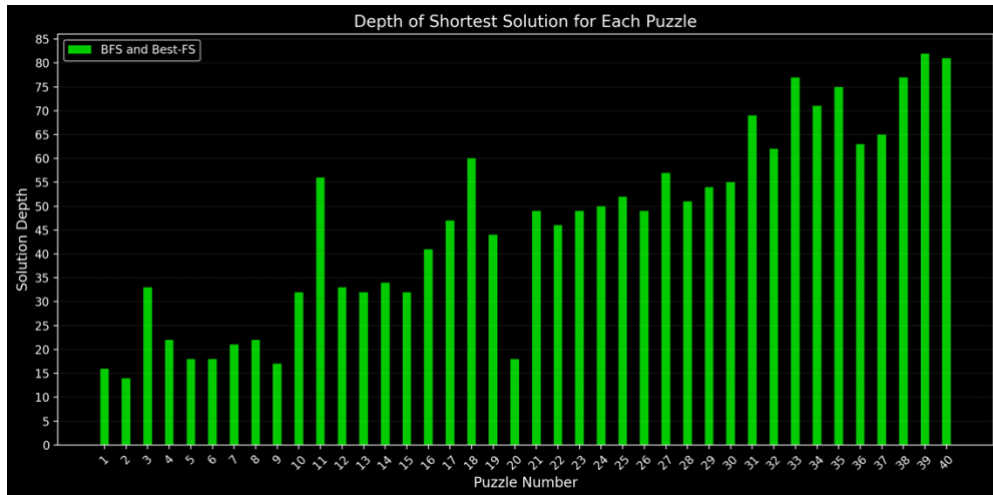


Figure 7: Depth of Shortest Solution for Each Board

Figure 7 illustrates the depth of the shortest solution for each board. Evidently there is a steady increase in the solution depths as problem difficulty increases. This is like the trend noticed in Figure 6 where the number of unique states visited steadily increases with problem difficulty. In addition, there are some boards that deviate from this trend which suggests that maybe certain board configurations demand more moves but are easier to solve. An example is Board 11, the solution depth is 55, the number of unique states visited is in the range 500 to 1000, and it clocks a low time (see figure 8).

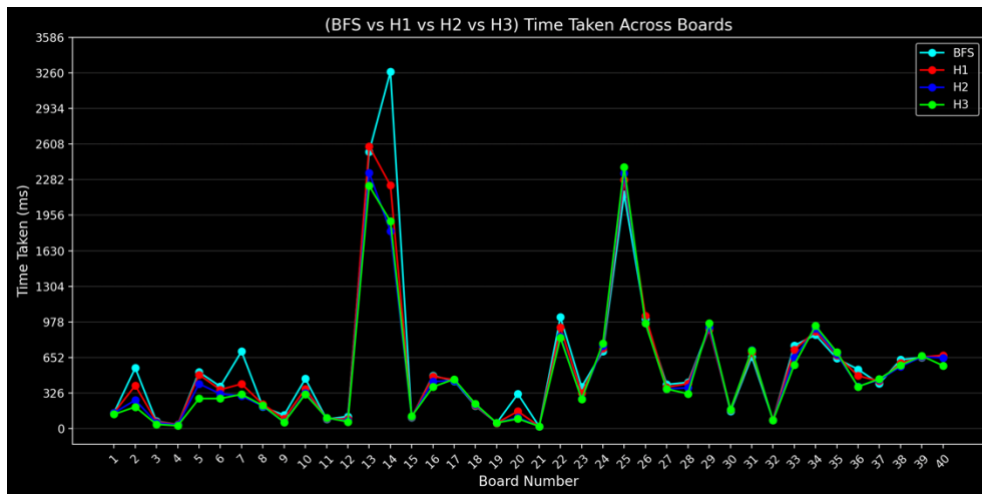


Figure 8: BFS vs H1 vs H2 vs H3) Time Taken Across Boards

Figure 8 shows the time taken for each algorithm (BFS, H1, H2, H3) across the 40 different boards. BFS generally takes the longest time. At the start (left side of the graph) H1, H2, H3 tend to perform better. H3 Performs the best up to board 23, after which it occasionally preforms equally or slightly worse than the other algorithms. This is likely due to the added complexity for the H3 estimation while the overall heuristics become less effective with increasing board difficulty (as pointed out earlier). Thus, taking more time to complete the search.

Conclusion

In conclusion, the analysis demonstrates how BFS and Best-FS-Heuristic algorithms perform while solving the Rush Hour Problem. BFS, as expected, explores the most states and takes the longest time due to its exhaustive nature. In contrast, heuristic-driven Best-FS algorithms reduce the number of states visited, with H3 being the most efficient in most cases. However, as search problem difficulty increases, the gap between BFS and the heuristics diminishes. This highlights the need for strong admissible heuristic estimations due to the limitations of heuristic guidance in deeper searches. The results from this project suggest that while heuristics can improve efficiency, their effectiveness varies depending on the problem complexity. This means it is key to study the problem and choose the right heuristic to guide the search.